

Challenge Desarrollador .NET

Sistema de Gestión de Turnos Médicos

Candidato:	Oscar Benjamin Villafañe Assef
DNI:	40533037
Fecha de entrega:	Mayo 2026
Stack:	.NET Core + Vue.js
Repositorio:	github.com/Benja-cmd/Challenge-Seniority

1. Introducción

El presente documento describe el trabajo realizado sobre el sistema de gestión de turnos médicos provisto como base del challenge. El trabajo comprende dos partes: la identificación y corrección de problemas en el código existente, y la implementación de la política de No-Show solicitada.

2. Parte 1 — Intervención del Código Existente

Se identificaron y corrigieron los siguientes problemas en el código base, organizados por archivo:

2.1 Modelos

Paciente.cs — Naming convention inconsistente

Las propiedades `createdAt` e `isActive` utilizaban camelCase en lugar de PascalCase, rompiendo la convención estándar de C#. Se renombraron a `CreatedAt` e `IsActive`.

Turno.cs — Campo faltante para política de No-Show

No existía registro de cuándo se realizó una cancelación. Se agregó la propiedad `FechaCancelacion` de tipo `DateTime?`, indispensable para determinar si una cancelación fue tardía (menos de 24 horas).

2.2 ApplicationDbContext — Datos de Seed Inconsistentes

Se detectaron dos inconsistencias en los datos de seed que generarían comportamientos incorrectos en la política de No-Show:

- **Paciente 2 (Mariana López):** tenía 2 turnos con estado NoShow en el seed de Turnos, pero su NoShowCount era 0. Se corrigió a NoShowCount = 2.

- **Paciente 7 (Facundo Sosa):** tenía Bloqueado = true pero NoShowCount = 0. Si un paciente está bloqueado por política de No-Show, su contador debe ser 3. Se corrigió a NoShowCount = 3.

2.3 Program.cs

Swagger expuesto en producción

Swagger estaba habilitado de forma incondicional. En un entorno productivo esto expone la documentación de la API públicamente. Se condicionó su activación al entorno de desarrollo:

```
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
```

CORS excesivamente permisivo

La política de CORS permitía cualquier origen, header y método (**AllowAnyOrigin**). En un sistema médico que maneja datos sensibles esto representa un riesgo de seguridad. Se restringió a los orígenes definidos en la configuración:

```
policy.WithOrigins(builder.Configuration.GetSection("AllowedOrigins").Get(!))
```

El origen permitido se define en **appsettings.json** bajo la clave **AllowedOrigins**, lo que permite modificarlo por entorno sin tocar código.

Sin middleware de manejo global de errores

No existía ningún mecanismo de manejo de excepciones no controladas. Cualquier error inesperado devolvería un 500 con el stack trace expuesto. Se documentó como punto de mejora fuera del scope dado el tiempo disponible.

2.4 TurnosController

CancelarTurno usaba HTTP GET para una operación de escritura

El endpoint de cancelación estaba decorado con **[HttpGet]**, violando los principios REST (GET debe ser idempotente y sin efectos secundarios). Se cambió a **[HttpPut]**.

Ventana de cancelación incorrecta (23h en lugar de 24h)

La comparación usaba 23 horas en lugar de las 24 horas que establece el requerimiento. Se corrigió el valor y se unificó el uso de **DateTime.UtcNow** en todos los puntos (el código original mezclaba **DateTime.Now** y **DateTime.UtcNow**).

Lógica de No-Show incompleta en CancelarTurno y MarcarAusencia

Ambos métodos modificaban el estado del turno pero no actualizaban al paciente. Se agregó la lógica de incremento de NoShowCount y bloqueo automático al alcanzar 3 ausencias.

CrearTurno no verificaba IsActive ni el vencimiento del bloqueo

El endpoint permitía crear turnos para pacientes dados de baja lógicamente. Tampoco verificaba si el bloqueo de 30 días había expirado, requiriendo intervención manual para desbloquear. Se agregaron ambas validaciones: rechazo si **IsActive = false**, y desbloqueo automático si pasaron 30 días desde **FechaBloqueo**.

ActualizarEstado sin validación de transiciones

El endpoint permitía cualquier cambio de estado, incluyendo transiciones inválidas como Atendido → Pendiente. Se implementó una matriz de transiciones válidas explícita.

2.5 PacientesController

Delete realizaba hard delete

El modelo ya contaba con el campo `IsActive` para soft delete, pero el endpoint lo ignoraba y eliminaba el registro físicamente. Esto rompe la integridad referencial con los turnos asociados. Se cambió a soft delete seteando `IsActive = false`.

GetAll y GetById retornaban pacientes inactivos

Al no filtrar por `IsActive`, los pacientes dados de baja seguían siendo visibles y operables. Se agregó el filtro en ambos endpoints.

Create no validaba DNI duplicado ni sanitizaba campos sensibles

Era posible crear dos pacientes con el mismo DNI. Además, el cliente podía enviar valores arbitrarios para `NoShowCount`, `Bloqueado` y `FechaBloqueo`. Se agregó validación de unicidad de DNI y se forzaron los valores de esos campos desde el servidor.

2.6 Helpers — DateTimeExtensions

El método `IsWithinCancellationWindow` usaba `DateTime.Now` en lugar de `DateTime.UtcNow`. Se refactorizó el helper en dos métodos con nombres más expresivos:

- `IsPasado()`: retorna true si la fecha del turno ya ocurrió.
- `EsCancelacionTardia()`: retorna true si quedan menos de 24 horas para el turno.

Estos métodos se utilizan en `TurnosService` para mantener la lógica de negocio legible y centralizada.

3. Refactor Arquitectónico — Capa de Servicios

El código base concentraba toda la lógica de negocio directamente en los controladores. Se introdujo una capa de servicios para separar responsabilidades, mejorar la testeabilidad y facilitar el mantenimiento. La estructura resultante es la siguiente:

Carpeta	Archivo	Responsabilidad
Services/Interfaces/	ITurnosService.cs	Contrato del servicio de turnos
Services/Interfaces/	IPacientesService.cs	Contrato del servicio de pacientes
Services/	TurnosService.cs	Lógica de negocio de turnos
Services/	PacientesService.cs	Lógica de negocio de pacientes
Controllers/	TurnosController.cs	Solo HTTP: delega en ITurnosService
Controllers/	PacientesController.cs	Solo HTTP: delega en IPacientesService

Los controladores ahora se limitan a recibir la request HTTP, delegar en el servicio y mapear las excepciones de negocio a códigos HTTP apropiados. Los servicios se registran con lifetime **Scoped** en `Program.cs`, adecuado para operaciones con `DbContext`.

4. Parte 2 — Implementación de Política de No-Show

4.1 Interpretación del Requerimiento

El requerimiento establece: "Si el paciente cancela con menos de 24 horas o no asiste, se marca. Con 3 ausencias, se bloquea por 30 días para agendar online."

Se identificaron dos escenarios que generan un No-Show:

- **Cancelación tardía:** el paciente cancela con menos de 24 horas de anticipación.
- **Ausencia:** el paciente no se presentó al turno (marcado manualmente por el sistema).

4.2 Flujo Implementado

Acción	Condición	Resultado
Cancelar turno	Más de 24h antes	Estado: Cancelado. Sin penalidad.
Cancelar turno	Menos de 24h antes	Estado: NoShow. NoShowCount++.
Marcar ausencia	Turno ya ocurrió	Estado: NoShow. NoShowCount++.
NoShowCount llega a 3	—	Bloqueado = true. FechaBloqueo = ahora.
Crear turno	Bloqueado y < 30 días	Rechazado con mensaje de bloqueo.
Crear turno	Bloqueado y >= 30 días	Desbloqueo automático. Turno creado.

4.3 Cambios en el Modelo de Datos

La política de No-Show utiliza campos ya presentes en el modelo Paciente:

- **NoShowCount** (int): contador de ausencias acumuladas.
- **Bloqueado** (bool): indica si el paciente está bloqueado.
- **FechaBloqueo** (DateTime?): fecha en que se aplicó el bloqueo, usada para calcular el vencimiento de 30 días.

Se agregó **FechaCancelacion** (DateTime?) al modelo Turno para registrar el momento exacto de la cancelación, permitiendo auditoría posterior.

5. Supuestos Tomados

- **Cancelación tardía = No-Show:** El enunciado dice 'si cancela con menos de 24h, se marca'. Se interpretó que la cancelación se permite pero el turno queda con estado NoShow, no Cancelado. No se bloquea la acción.
- **Desbloqueo automático al cumplir 30 días:** El enunciado no especifica si el desbloqueo es manual o automático. Se optó por desbloquearlo en el momento en que el paciente intenta crear un nuevo turno y ya pasaron 30 días, evitando la necesidad de un proceso batch o job programado.
- **NoShowCount no se reinicia al desbloquear:** Al cumplir los 30 días y desbloquearse, el NoShowCount se resetea a 0 para dar una segunda oportunidad limpia al paciente, ya que el bloqueo fue su consecuencia y ya fue cumplido.

- **Ausencia solo marcabable en turnos pasados:** Solo se permite marcar ausencia en turnos cuya FechaHora ya ocurrió, para evitar marcar como ausente a un paciente que aún tiene tiempo de presentarse.
- **Médicos y Sucursales sin capa de servicio:** Dado que ambos controllers solo tienen un GET sin lógica de negocio, no se justifica agregar una capa de servicio. Se documentó como decisión de diseño consciente.

6. Decisiones Fuera del Scope

- **Autenticación y autorización:** El sistema no implementa ningún mecanismo de autenticación. En un sistema médico real esto sería crítico, pero excede el alcance del challenge.
- **Middleware global de excepciones:** Se identificó la ausencia de un `ExceptionHandler` global. Se priorizó la corrección de bugs de negocio y la implementación de la feature. Se podría agregar con `UseExceptionHandler`.
- **Validaciones con `FluentValidation` o `DataAnnotations`:** Los modelos no tienen anotaciones de validación. Se priorizó la lógica de negocio sobre la validación de inputs para optimizar el tiempo disponible.
- **Tests unitarios:** La arquitectura con interfaces e inyección de dependencias facilita la escritura de tests, pero no se implementaron dado el tiempo límite de 72 horas.
- **Notificaciones al paciente:** En un sistema real, el bloqueo debería notificarse al paciente por email o SMS. Se dejó fuera del scope por no estar en el requerimiento.

7. Refactor — Introducción de DTOs

El código base exponía y recibía directamente las entidades del dominio en los endpoints. Esto genera dos problemas: el cliente puede enviar campos que no debería manipular (como `NoShowCount` o `Bloqueado`), y la API queda acoplada a la estructura interna del modelo. Se introdujo una capa de DTOs separando claramente lo que entra y lo que sale:

7.1 Requests — lo que recibe la API

Archivo	Campos expuestos
<code>CrearTurnoRequest</code>	<code>PacienteId</code> , <code>MedicoId</code> , <code>FechaHora</code> , <code>Motivo</code>
<code>ActualizarEstadoRequest</code>	<code>Estado</code>
<code>CrearPacienteRequest</code>	<code>NombreCompleto</code> , <code>DNI</code> , <code>Email</code> , <code>Telefono</code>
<code>ActualizarPacienteRequest</code>	<code>NombreCompleto</code> , <code>DNI</code> , <code>Email</code> , <code>Telefono</code>

7.2 Responses — lo que devuelve la API

Archivo	Campos expuestos	Campos excluidos
---------	------------------	------------------

TurnoResponse	Id, PacienteId, PacienteNombre, MedicoId, MedicoNombre, MedicoEspecialidad, FechaHora, Estado, FechaCreacion, FechaCancelacion, Motivo	—
PacienteResponse	Id, NombreCompleto, DNI, Email, Telefono, Bloqueado, NoShowCount, CreatedAt	IsActive, FechaBloqueo

El mapeo de entidad a Response se realiza mediante un método privado estático MapToResponse() en cada servicio, evitando duplicación y manteniendo la transformación centralizada.

8. Resumen de Archivos Modificados y Creados

Archivo	Tipo	Descripción del cambio
Models/Paciente.cs	Modificado	Naming convention PascalCase
Models/Turno.cs	Modificado	Agregado campo FechaCancelacion
Data/AppDbContext.cs	Modificado	Corrección seed NoShowCount pacientes 2 y 7
Helpers/DateTimeExtensions.cs	Modificado	Refactor: IsPasado() y EsCancelacionTardia()
Controllers/TurnosController.cs	Modificado	Usa DTOs, delega en ITurnosService
Controllers/PacientesController.cs	Modificado	Usa DTOs, soft delete, delega en IPacientesService
Services/Interfaces/ITurnosService.cs	Creado	Contrato de servicio de turnos con DTOs
Services/Interfaces/IPacientesService.cs	Creado	Contrato de servicio de pacientes con DTOs
Services/TurnosService.cs	Creado	Lógica de negocio + política No-Show + mapeo DTOs
Services/PacientesService.cs	Creado	Lógica de negocio de pacientes + mapeo DTOs
DTOs/Requests/CrearTurnoRequest.cs	Creado	Input para creación de turno
DTOs/Requests/ActualizarEstadoRequest.cs	Creado	Input para actualización de estado
DTOs/Requests/CrearPacienteRequest.cs	Creado	Input para creación de paciente
DTOs/Requests/ActualizarPacienteRequest.cs	Creado	Input para actualización de paciente
DTOs/Responses/TurnoResponse.cs	Creado	Output de endpoints de turnos
DTOs/Responses/PacienteResponse.cs	Creado	Output de endpoints de pacientes
Program.cs	Modificado	Registro de servicios, CORS restringido, Swagger condicional
appsettings.json	Modificado	Agregado AllowedOrigins

9. Frontend — Correcciones Vue.js

Se revisaron todos los archivos del frontend Vue.js y se corrigieron los siguientes problemas:

9.1 Services/api.js

- **cancelar usaba HTTP GET en lugar de PUT:** el método cancelar llamaba a axios.get en lugar de axios.put, inconsistente con el fix realizado en el backend. Se corrigió a axios.put con la ruta correcta /turnos/{id}/cancelar.

- **BASE_URL hardcodeada a servidor de producción:** la URL apuntaba directamente al servidor de la empresa. Se reemplazó por `import.meta.env.VITE_API_URL` y se creó el archivo `.env.development` con la URL local del backend.

9.2 TurnosList.vue

- **Campos de objetos anidados reemplazados por campos planos del DTO:** `turno.paciente?.nombreCompleto` se reemplazó por `turno.pacienteNombre`, `turno.medico?.nombreCompleto` por `turno.medicoNombre`, `turno.medico?.especialidad` por `turno.medicoEspecialidad`.
- **cancelar sin try/catch ni feedback:** si la cancelación fallaba no se mostraba ningún error. Se agregó `try/catch` con mensaje del backend y confirmación previa.
- **Lista no se actualizaba tras cancelar:** el turno cancelado no reflejaba el nuevo estado en la tabla. Se actualiza la fila con la respuesta del servidor.
- **Botón cancelar sin validación de estado:** se podía intentar cancelar un turno ya cancelado o atendido. Se deshabilita el botón según el estado actual.

9.3 TurnoDetalle.vue

- **Campos planos del DTO:** misma corrección que en `TurnosList`. Se removió también el campo `DNI` que ya no forma parte de `TurnoResponse`.
- **cancelar y marcarAusencia sin try/catch ni actualización de estado:** ambos métodos no manejaban errores ni refrescaban el turno en pantalla. Se agregó `try/catch`, confirmación y actualización con la respuesta del servidor.
- **Select de estados mostraba todas las opciones:** permitía seleccionar transiciones inválidas. Se reemplazó por un `computed` `estadosDisponibles` que refleja las transiciones válidas según el estado actual.
- **Botones y sección de cambio de estado visibles en estados finales:** se ocultaron con `v-if` y se deshabilitaron botones cuando el turno ya no es operable.
- **fechaCancelacion no se mostraba:** se agregó una fila que muestra la fecha de cancelación cuando el campo está presente.

9.4 TurnoNuevo.vue

- **Sin validación de campos vacíos:** enviaba `patientId`: 0 si no se seleccionaba paciente o médico. Se agregó validación con mensajes de error por campo.
- **Fecha sin restricción de fecha mínima:** era posible seleccionar fechas pasadas. Se agregó el atributo `min` con la fecha/hora actual via `computed` `fechaMinima`.
- **Uso de etiqueta form con submit:** se reemplazó por manejo directo con `@click` en el botón para mayor consistencia con el resto del proyecto.

9.5 PacientesList.vue

- **Eliminar sin confirmación:** un click accidental eliminaba al paciente sin ninguna advertencia. Se agregó `confirm()` antes de ejecutar la acción.

- **Mensaje de error genérico:** se reemplazó el mensaje genérico por el mensaje específico devuelto por el backend cuando está disponible.

Archivo	Tipo	Descripción del cambio
Services/api.js	Modificado	Fix HTTP GET a PUT en cancelar, BASE_URL por variable de entorno
.env.development	Creado	VITE_API_URL apuntando al backend local
Views/TurnosList.vue	Modificado	Campos DTO planos, try/catch, actualización UI, botón deshabilitado
Views/TurnoDetalle.vue	Modificado	Campos DTO planos, try/catch, estados válidos, fechaCancelacion
Views/TurnoNuevo.vue	Modificado	Validación de campos, fecha mínima, manejo de errores
Views/PacientesList.vue	Modificado	Confirmación antes de eliminar, mensajes de error del backend